



《AI大模型Ragent项目》——用户问退货政策，3C、家电和服装都举手了

来自： 拿个offer-开源&项目实战

 马丁

2026年04月27日 22:29

开篇引言

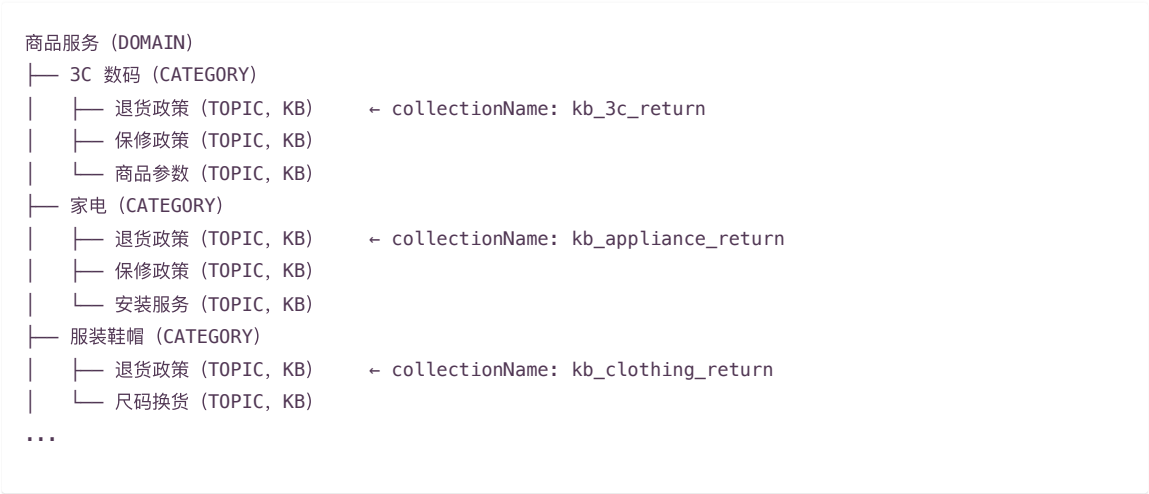
上一篇讲了总量封顶算法——三个子问题命中八个意图，capTotalIntents 用多样性优先 + 分数竞争的策略把意图总数压到 3 个以内。到这里，每个子问题至少有一个代表意图，数量可控，下游检索不会被意图数量拖垮。

封顶解决了数量太多的问题，但有一种场景它管不了：**几个意图分数差不多，名字还一样，分属不同品类**。封顶算法只看分数和子问题归属，不关心意图之间是不是同名、是不是来自不同品类。它选出来的 3 个意图可能恰好是三个品类的退货政策——分数都在 0.6 附近，你取哪个都不踏实。

回到第 5 篇埋下的伏笔。那篇展示的电商意图树有 11 个叶子节点，对应一个品类不多的中型电商。文章末尾提了一句：如果再加上不同品类（3C 数码、家电、服装）各自的专属知识库，轻轻松松到 20 个。

现在把这个伏笔接上。

业务会长大。当平台从综合电商发展到多品类运营，原来一个笼统的退换货知识库就不够用了——3C 数码有 15 天无理由退货加厂商质保，家电有 30 天退换加上门安装，服装有 7 天无理由加尺码换货。退货政策、保修政策、售后流程全都按品类拆分了。意图树也跟着长：



三个品类的 CATEGORY 下面都有一个叫退货政策的 TOPIC 叶子节点，但绑定的 Collection 各不相同。3C 的退货政策、家电的退货政策、服装的退货政策是三个独立知识库。

这时用户在对话框里输入：

退货政策是什么？

没指定品类。意图分类器诚实地返回了三个分数接近的意图：

- 3C 数码 > 退货政策 (0.68)
- 家电 > 退货政策 (0.65)
- 服装鞋帽 > 退货政策 (0.62)

放个实际效果图：

退换货规则

关于退换货规则，在知识库中检索到了以下内容：

1. 家电 > 退换货规则

2. 电商 > 囤换货 > 退换货规则

3. 3C数码 > 退换货 > 退货政策

请问你具体想了解哪个？请回复数字选择（可多选，如 1,2），或回复“都/全部”

如果直接取最高分走 3C 的知识库，万一用户买的是家电呢？如果三个都搜，答案里 3C、家电、服装的退货规则混在一起，15 天、30 天、7 天三种政策并排放，核心信息被稀释，用户反而更懵。

最合理的做法是停下来问一句：**你看哪个品类的退货政策？**

这就是本篇要讲的——歧义检测与引导式问答。IntentGuidanceService 负责识别跨品类的歧义场景，生成引导文案，通过 SSE 短路输出给用户。多数情况下纯内存计算即可判定，只有分数落在灰色地带时才调一次 LLM 做二次确认，整个过程不走检索，前端零适配。

什么时候算歧义：规则 + LLM 分层判定

1. 歧义的直觉定义

用一句话讲清楚：**多个品类的意图分数都很接近，系统无法确定用户想问哪个品类，就是歧义场景。**

什么情况下不算歧义？看几个反例：

场景	为什么不算歧义
3C 退货政策（0.88）+ 家电退货政策（0.35），分数差距大	0.88 vs 0.35 一目了然，直接走 3C 没毛病
用户问"3C 数码的退货政策"，分数接近	用户已经明示了品类，不需要再问
3C 线上退货政策（0.68）+ 3C 线下退货政策（0.65），都在同一品类下	同一品类内部的节点，下游检索走同一个品类知识库，结果是相关的

真正需要引导的场景是：分数接近 + 跨品类 + 用户没有指定品类。

2. findAmbiguityGroup 代码总览

歧义检测的核心逻辑在 IntentGuidanceService 的 findAmbiguityGroup() 方法里。先看完整代码，再逐段拆解。

```
private AmbiguityGroup findAmbiguityGroup(String question, List<SubQuestionIntent> subIntents) {
    if (CollUtil.isEmpty(subIntents) || subIntents.size() != 1) {
        return null;
    }

    List<NodeScore> candidates = filterCandidates(subIntents.get(0).nodeScores());
    if (candidates.size() < 2) {
        return null;
    }

    // 按品类分组，每个品类保留最高分的 topic
    Map<String, NodeScore> systemBest = candidates.stream()
        .filter(ns -> StrUtil.isNotBlank(resolveSystemNodeId(ns.getNode())))
        .collect(Collectors.toMap(
            ns -> resolveSystemNodeId(ns.getNode()),
            ns -> ns,
            (a, b) -> a.getScore() >= b.getScore() ? a : b
        ));

    List<NodeScore> ranked = systemBest.values().stream()
        .sorted(Comparator.comparingDouble(NodeScore::getScore).reversed())
        .toList();

    if (ranked.size() < 2) {
        return null;
    }

    // 快速跳过：分数差距大 或 用户已指定系统
    if (shouldSkipGuidance(question, ranked)) {
        return null;
    }

    // 三区间判定：明确歧义 / 灰色地带调 LLM / 明确无歧义
    if (!confirmAmbiguity(question, ranked)) {
        return null;
    }
}
```

```
List<NodeScore> trimmedRanked = trimRankedOptions(ranked);
String topicName = trimmedRanked.get(0).getNode().getName();
return new AmbiguityGroup(topicName, trimmedRanked);
}

private record AmbiguityGroup(String topicName, List<NodeScore> ranked) {
}
```

五个步骤：

- 1. 前置过滤：只处理单子问题场景，KB 类候选至少 2 个
- 2. 按品类分组：通过 resolveSystemNodeId 找到每个意图的品类归属，每个品类只保留最高分的 topic 作为代表
- 3. 快速跳过：分数差距明显或用户已指定系统时，直接判定不歧义
- 4. 三区间判定：根据分数比值所在区间，决定是明确歧义、调 LLM 确认还是明确无歧义
- 5. 转换为 AmbiguityGroup：提取主题名 + 品类级选项 ID 列表

AmbiguityGroup 是一个 record，结构很简单——topicName 是触发歧义的那个主题名称（退货政策），ranked 是按分数降序排列的品类代表列表，包含完整的节点信息（名称、路径、分数），方便下游渲染选项时直接使用。

3. 前置过滤：只处理单子问题

```
if (CollUtil.isEmpty(subIntents) || subIntents.size() != 1) {
    return null;
}
```

subIntents.size() != 1 直接返回 null——只有单子问题才做歧义判断。
为什么？回忆一下第 4 篇查询改写和第 7 篇封顶算法。复合问题已经被拆成了多个子问题，每个子问题独立走意图分类。子问题 A 命中退货政策、子问题 B 命中物流费规则，这不是歧义，是用户确实问了两件事。跨子问题去做歧义判断没有意义——它们本来就该是不同意图。过了这一关，说明是单子问题。但单子问题如果只命中了 1 个意图，也没有可比的对象，不可能构成歧义。所以紧接着还要检查候选数量：

```
List<NodeScore> candidates = filterCandidates(subIntents.get(0).nodeScores());
if (candidates.size() < 2) {
    return null;
}
```

filterCandidates() 做了什么？只保留 KB 类意图，且分数 >= 0.35（INTENT_MIN_SCORE）：

```
private List<NodeScore> filterCandidates(List<NodeScore> scores) {
    if (CollUtil.isEmpty(scores)) {
        return List.of();
    }
    return NodeScoreFilters.kb(scores, RAGConstant.INTENT_MIN_SCORE);
}
```

MCP 意图和 SYSTEM 意图不参与歧义引导。MCP 工具是外部接口调用（比如订单查询），不存在跨品类的概念；SYSTEM 意图是打招呼之类的系统交互，更不需要歧义引导。歧义引导是知识库场景的事。

按品类分组：每个品类取最强代表

1. 为什么按品类分组

旧的思路是按节点名称分组——找出名字一样的节点，再检查是否跨品类。但这有一个刚性限制：管理员 A 写退货政策、管理员 B 写退换货规则，语义相同但名字不同，按名称分组就把它们分开了，检测不出歧义。
新方案换了个思路：**先按品类分组，每个品类只保留分数最高的 topic 作为代表，然后看各品类代表之间的分数是否接近。**不再要求节点名称相同——反正最终引导用户选的是品类，不是具体 topic。

```
Map<String, NodeScore> systemBest = candidates.stream()
    .filter(ns -> StrUtil.isNotBlank(resolveSystemNodeId(ns.getNode())))
```

```
.collect(Collectors.toMap(
    ns -> resolveSystemNodeId(ns.getNode()),
    ns -> ns,
    (a, b) -> a.getScore() >= b.getScore() ? a : b
));
```

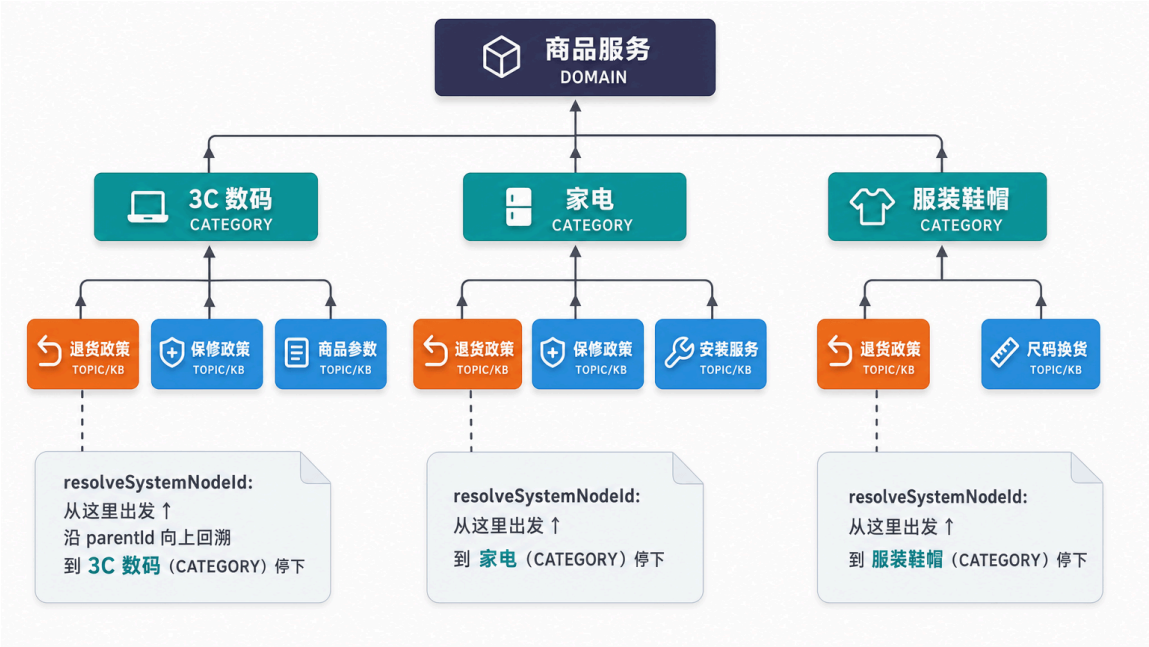
Collectors.toMap 的第三个参数是合并函数——同一品类下有多个候选 topic 时（比如 3C 数码下既有退货政策又有保修政策），保留分数高的那个。每个品类只出一个代表。
分完组之后按分数降序排列：

```
List<NodeScore> ranked = systemBest.values().stream()
    .sorted(Comparator.comparingDouble(NodeScore::getScore).reversed())
    .toList();
```

ranked.get(0) 就是得分最高的品类代表，ranked.get(1) 是次高的。如果品类代表不足 2 个（比如所有候选都属于同一品类），直接返回 null——同品类内部不存在歧义。

2. resolveSystemNodeId 的向上回溯

品类分组的关键在于 resolveSystemNodeId——它从叶子节点出发，沿 parentId 向上回溯，找到 CATEGORY 级祖先作为品类归属。回顾第 5 篇的三级意图图树，扩展后的电商品类分支长这样：



三级结构各自的粒度：

层级	电商场景的含义	作为品类归属的问题
DOMAIN（商品服务）	最粗的业务板块	3C、家电、服装都在商品服务下面，区分不出品类
CATEGORY（3C 数码 / 家电 / 服装鞋帽）	品类粒度	刚好是想要的品类维度
TOPIC（退货政策）	最细的知识方向	就是触发歧义的那个节点本身，不是品类归属

所以 resolveSystemNodeId 在 CATEGORY 级停下——这一级刚好是品类的粒度，DOMAIN 太粗（所有品类都在下面），TOPIC 太细（是节点本身）。

```
private String resolveSystemNodeId(IntentNode node) {
    if (node == null) {
        return "";
    }
    IntentNode current = node;
    IntentNode parent = fetchParent(current);
    for (; ; ) {
        IntentLevel level = current.getLevel();
```

```
    if (level == IntentLevel.CATEGORY && (parent == null || parent.getLevel() == IntentLevel.D
        return current.getId();
    }
    if (parent == null) {
        return current.getId();
    }
    current = parent;
    parent = fetchParent(current);
}

private IntentNode fetchParent(IntentNode node) {
    if (node == null || StrUtil.isBlank(node.getParentId())) {
        return null;
    }
    return intentNodeRegistry.getNodeById(node.getParentId());
}
```

用退货政策场景走一遍：

1. current = 退货政策 (TOPIC), parent = 3C 数码 (CATEGORY)
2. current.getLevel() 是 TOPIC, 不满足终止条件
3. current 上移到 3C 数码 (CATEGORY), parent 上移到商品服务 (DOMAIN)
4. current.getLevel() 是 CATEGORY, parent.getLevel() 是 DOMAIN, 满足终止条件
5. 返回 3C 数码的节点 ID

三个退货政策节点分别返回 3C 数码、家电、服装鞋帽的 ID，分到三个不同的品类组里。

fetchParent 通过 intentNodeRegistry.getNodeById() 根据 parentId 查注册表拿父节点。注册表在启动时加载意图树数据后构建，运行时走内存查询，不涉及数据库 IO。

快速跳过：shouldSkipGuidance

品类分好组、排好序之后，先走两个快速通道判断是否可以直接跳过歧义引导。

1. 两条快速通道

```
private boolean shouldSkipGuidance(String question, List<NodeScore> ranked) {
    double top = ranked.get(0).getScore();
    if (top <= 0) {
        return true;
    }

    // 快速通道 1：分数比值低于边界下限，意图明确
    double ratio = ranked.get(1).getScore() / top;
    double threshold = Optional.ofNullable(guidanceProperties.getAmbiguityScoreRatio()).orElse(0.8)
    double margin = Optional.ofNullable(guidanceProperties.getAmbiguityMargin()).orElse(0.15D);
    if (ratio < threshold - margin) {
        return true;
    }

    // 快速通道 2：用户问题中显式提到了某个系统的 DOMAIN 级名称
    if (StrUtil.isNotBlank(question)) {
        List<String> domainNames = ranked.stream()
            .map(ns -> resolveDomainName(ns.getNode()))
            .filter(StrUtil::isNotBlank)
            .distinct()
            .toList();

        String normalizedQuestion = normalizeName(question);
```

```
for (String name : domainNames) {
    for (String alias : buildSystemAliases(name)) {
        if (alias.length() >= 2 && normalizedQuestion.contains(alias)) {
            return true;
        }
    }
}
return false;
}
```

两条通道分别拦截两种不需要引导的情况。

2. 通道一：分数差距大，直接走最高分

```
double ratio = ranked.get(1).getScore() / top;
if (ratio < threshold - margin) {
    return true;
}
```

threshold 默认 0.8, margin 默认 0.15。两者相减得到**边界下限** 0.65。当次高分 / 最高分 < 0.65 时，说明最高分品类有压倒性优势，直接走它就行，不需要引导。

为什么用比值而不是差值？高分段和低分段的含义不同：

0.88 vs 0.78, 差 0.1, 但 0.88 那个明显更好，不该算歧义

0.45 vs 0.35, 差 0.1, 两个都不太行，也不该触发引导

比值法考虑了分数的量级。0.65 / 0.68 ≈ 0.96, 远超 0.65 下限，说明两个分数确实非常接近，不能跳过。0.35 / 0.88 ≈ 0.40, 远低于 0.65 下限，说明差距很大，直接跳过。

3. 通道二：用户已指定系统

这条通道面向的是多系统架构——比如企业同时接入了 OA 系统、保险系统、电商系统，每个系统对应意图树的一个 DOMAIN 级节点。考虑两种情况：

场景 A：用户问“数据安全怎么管理？”——没指定系统，OA 和保险都有数据安全相关节点，分数接近，确实需要引导

场景 B：用户问“OA系统的数据安全管理”——已经说了 OA 系统

场景 B 即使分数接近，也不该弹引导——用户已经明示了系统归属。

resolveDomainName 从节点向上追溯到 DOMAIN 级祖先，获取系统名称：

```
private String resolveDomainName(IntentNode node) {
    if (node == null) {
        return "";
    }
    IntentNode current = node;
    while (current != null) {
        if (current.getLevel() == IntentLevel.DOMAIN) {
            return StrUtil.blankToDefault(current.getName(), "");
        }
        current = fetchParent(current);
    }
    return "";
}
```

拿到 DOMAIN 名称后，做归一化字符串匹配。buildSystemAliases 目前只返回归一化后的主名，返回 List 留了扩展点，未来可以加别名字典。

注意：这条通道匹配的是 DOMAIN 级名称（系统级），不是 CATEGORY 级名称（品类级）。对于单域名多品类的电商场景（所有品类都在同一个“商品服务”DOMAIN 下），这条通道通常不会命中。品类级的语义消歧（比如“手机”属于 3C 数码）交给后面的 LLM 确认环

节处理。

4. 名称归一化

快速通道二依赖 `normalizeName()` 做字符串匹配。归一化逻辑：

```
private String normalizeName(String name) {
    if (name == null) {
        return "";
    }
    String cleaned = name.trim().toLowerCase(Locale.ROOT);
    return cleaned.replaceAll("[\\p{Punct}\\s]+", "");
}
```

三步走：

- `trim()`：去首尾空格
- `toLowerCase(Locale.ROOT)`：统一小写。用 `Locale.ROOT` 而不是 `Locale.getDefault()` 是有讲究的——在土耳其 `Locale` 下，`I.toLowerCase()` 变成的不是 `i` 而是 `ı`（一个没有点的 `i`），会导致匹配不上。`Locale.ROOT` 保证行为和 `Locale` 设置无关
- 正则** `[\\p{Punct}\\s]+`：去除所有 Unicode 标点和空白字符

归一化效果：

原始名称	归一化后
退货政策	退货政策
退货 政策	退货政策
退货-政策	退货政策
Return Policy	returnpolicy

5. 完整演算

场景 B：用户问“OA系统的数据安全管理”

- 归一化问题：`normalizeName("OA系统的数据安全管理")` → `"oa系统的数据安全管理"`
- 候选 DOMAIN 名称（去重后）：`["OA系统", "保险系统"]`
- 遍历 DOMAIN 名称：
 - `normalizeName("OA系统")` → `"oa系统"`，长度 `4 >= 2`
 - `"oa系统的数据安全管理".contains("oa系统")` → `true`
- 返回 `true`，跳过引导

场景 A：用户问“数据安全怎么管理？”

- 归一化问题：数据安全怎么管理
- 遍历 DOMAIN 名称：
 - `"数据安全怎么管理".contains("oa系统")` → `false`
 - `"数据安全怎么管理".contains("保险系统")` → `false`
- 全部 `false`，返回 `false`，不跳过，进入下一步三区间判定

电商单域名场景：用户问“3C 数码的退货政策”

- 候选 DOMAIN 名称（去重后）：`["商品服务"]`（三个品类都属于同一个 DOMAIN）
- `"3c数码的退货政策".contains("商品服务")` → `false`
- 字符串匹配不命中，但不用担心——如果分数差距足够大（通道一拦截），或者落在灰色地带（LLM 确认 3C 数码 是明确的品类线索，判定不歧义），系统都能正确处理

三区间判定：confirmAmbiguity

快速跳过没拦住，说明分数比值在 [threshold - margin, 1.0] 区间内，且用户问题里没有显式品类名。接下来要做正式的歧义判定。

1. 三个区间

```
private boolean confirmAmbiguity(String question, List<NodeScore> ranked) {
    double top = ranked.get(0).getScore();
    double second = ranked.get(1).getScore();
    if (top <= 0) {
        return false;
    }

    double ratio = second / top;
    double threshold = guidanceProperties.getAmbiguityScoreRatio();
    double margin = guidanceProperties.getAmbiguityMargin();

    if (ratio >= threshold) {
        // 明确歧义，直接触发引导
        return true;
    }

    if (ratio >= threshold - margin) {
        // 灰色地带，调 LLM 二次确认
        return ambiguityLLMChecker.checkAmbiguity(question, ranked);
    }

    // ratio < threshold - margin, 不歧义（理论上不会走到这里，shouldSkipGuidance 已拦截）
    return false;
}
```

以默认配置 threshold = 0.8、margin = 0.15 为例，分数比值被划分为三个区间：

区间	比值范围	判定结果	说明
明确歧义	ratio >= 0.8	直接触发引导	两个品类分数非常接近，系统没有信心做选择
灰色地带	0.65 <= ratio < 0.8	调 LLM 确认	有一定差距但不够大，需要语义理解辅助判断
明确无歧义	ratio < 0.65	不触发	最高分品类优势明显（已在 shouldSkipGuidance 拦截）

2. 为什么设灰色地带

纯规则方案有个两难：

 阈值设 0.8——比值 0.79 的场景不触发，但用户确实在两个品类之间摇摆

 阈值设 0.7——比值 0.72 的场景也触发，但最高分品类其实挺明确的

灰色地带的引入解决了这个问题。margin = 0.15 在阈值下方划出一个 [0.65, 0.8) 的缓冲区：

 比值 0.79：落在灰色地带，交给 LLM 判断。如果用户问“我买了个手机想退货”，LLM 能理解手机属于 3C 数码，返回不歧义

 比值 0.96：明确歧义区间，不需要调 LLM，规则直接判定

 比值 0.55：明确无歧义区间，shouldSkipGuidance 已经拦截

这样既保持了明确场景的零延迟判定，又在边界场景借助 LLM 的语义理解能力提升准确度。

3. 配置类

```
@Data
@Configuration
@ConfigurationProperties(prefix = "rag.guidance")
public class GuidanceProperties {
```



```
private Boolean enabled = true;
private Double ambiguityScoreRatio = 0.8D;
private Double ambiguityMargin = 0.15D;
private Integer maxOptions = 6;
}
```

三个参数都可以通过 `rag.guidance.*` 按业务调优。`margin` 越大，灰色地带越宽，调 LLM 的概率越高（准确度高但延迟和成本也高）；`margin` 设为 0 则退化为纯规则方案。

LLM 歧义确认器：AmbiguityLLMChecker

1. 整体设计

灰色地带的场景交给 `AmbiguityLLMChecker` 做二次确认。它的职责很简单：把用户问题和候选品类列表交给大模型，问一句这算不算歧义，拿回 JSON 结果。

```
@Component
@RequiredArgsConstructor
public class AmbiguityLLMChecker {

    private final LLMService llmService;
    private final PromptTemplateLoader promptTemplateLoader;

    public boolean checkAmbiguity(String question, List<NodeScore> ranked) {
        String candidatesText = buildCandidatesText(ranked);
        String prompt = promptTemplateLoader.render(
            GUIDANCE_AMBIGUITY_CHECK_PROMPT_PATH,
            Map.of("question", question, "candidates", candidatesText)
        );

        ChatRequest request = ChatRequest.builder()
            .messages(List.of(ChatMessage.user(prompt)))
            .temperature(0.1D)
            .topP(0.3D)
            .thinking(false)
            .build();

        try {
            String raw = llmService.chat(request);
            String cleaned = LLMResponseCleaner.stripMarkdownCodeFence(raw);
            JsonObject obj = JsonParser.parseString(cleaned).getAsJsonObject();
            return obj.get("ambiguous").getAsBoolean();
        } catch (Exception e) {
            log.warn("歧义确认 LLM 调用失败，降级为触发引导", e);
            return true;
        }
    }
}
```

几个设计要点：

- 低 Temperature (0.1) + 低 Top-P (0.3)：**歧义判断需要确定性输出，不需要创造力
- 关闭 thinking：**不需要 CoT 推理，只需要一个布尔判断
- 降级策略：**LLM 调用失败时返回 `true`（触发引导）。宁可多问一句，不可误走错品类

2. Prompt 设计

模板文件 `guidance-ambiguity-check.st`：

用户问题: {question}

以下是意图分类的候选品类及其匹配分数:
{candidates}

请判断:
1. 用户的问题是否存在品类歧义 (即无法确定用户想问哪个品类) ?
2. 如果存在歧义, 列出需要让用户选择的品类 ID; 如果不存在歧义, 给出最匹配的品类 ID。

注意:
- 如果用户问题中包含明确的领域线索 (如"手机"属于 3C 数码、"保单"属于保险), 则不算歧义
- 如果两个品类名称虽然不同但语义相同 (如"退货政策"和"退换货规则"), 且用户未指明具体品类, 则算歧义

以 JSON 格式输出:
{
 "ambiguous": true/false,
 "category_ids": ["最匹配或需要选择的品类ID"],
 "reason": "判断理由"
}

Prompt 里的两条注意事项精确覆盖了规则方案处理不好的场景:

- 领域线索识别: 我买了个手机想退货——LLM 能理解手机属于 3C 数码, 判定不歧义, 避免了字符串匹配的局限
- 同义词识别: 退货政策和退换货规则——LLM 能理解它们是同一件事, 即使归一化后字符串不同, 也能正确判定为歧义

candidates 占位符填充的是结构化的品类信息:

```
private String buildCandidatesText(List<NodeScore> ranked) {
    return ranked.stream()
        .map(ns -> {
            IntentNode node = ns.getNode();
            String systemPath = node.getFullPath() != null ? node.getFullPath() : node.getName
            return String.format("- 品类ID: %s, 名称: %s, 路径: %s, 分数: %.2f",
                                node.getId(), node.getName(), systemPath, ns.getScore());
        })
        .collect(Collectors.joining("\n"));
}
```

给 LLM 提供了品类 ID、名称、完整路径和分数, 信息足够做出判断。

3. 为什么降级策略是触发引导

LLM 调用可能因为网络超时、模型过载等原因失败。降级时有两个选择:

- 返回 false (不触发引导): 用户问题可能真的有歧义, 系统强行走最高分品类, 给出错误答案
- 返回 true (触发引导): 多问一句, 用户多点一下, 但不会给错答案

引导的代价是多一轮交互, 不引导的代价可能是答非所问。从用户体验角度, 宁可多问不可答错, 所以降级策略选择触发引导。

歧义检测入口: detectAmbiguity

看完了内部逻辑, 回到对外暴露的入口方法。它的结构很简洁:

```
public GuidanceDecision detectAmbiguity(String question, List<SubQuestionIntent> subIntents) {
    if (!Boolean.TRUE.equals(guidanceProperties.getEnabled())) {
        return GuidanceDecision.none();
    }

    AmbiguityGroup group = findAmbiguityGroup(question, subIntents);
    if (group == null || CollUtil.isEmpty(group.ranked())) {
        return GuidanceDecision.none();
    }

    String prompt = buildPrompt(group.topicName(), group.ranked());
}
```

```
return GuidanceDecision.prompt(prompt);
}
```

三步判断：

- 1. **总开关**：rag.guidance.enabled=false 时整体关闭引导功能
- 2. **歧义组检测**：findAmbiguityGroup() 内部完成品类分组、快速跳过、三区间判定的全部逻辑，返回 null 说明不存在歧义
- 3. **构造引导 Prompt**：通过模板渲染出引导文案

返回值 GuidanceDecision 是一个二态结构（直译引导决策，表示是否需要引导用户）：

```
@Getter
public class GuidanceDecision {

    public enum Action {
        NONE,
        PROMPT
    }

    private final Action action;
    private final String prompt;

    private GuidanceDecision(Action action, String prompt) {
        this.action = action;
        this.prompt = prompt;
    }

    public static GuidanceDecision none() {
        return new GuidanceDecision(Action.NONE, null);
    }

    public static GuidanceDecision prompt(String prompt) {
        return new GuidanceDecision(Action.PROMPT, prompt);
    }

    public boolean isPrompt() {
        return action == Action.PROMPT;
    }
}
```

要么 NONE（不引导，继续下游流程），要么 PROMPT（带着引导文案短路）。工厂方法 none() 和 prompt() 保证对象只能通过这两种方式创建。下游 Pipeline 只需要调一个 isPrompt() 就知道该怎么处理，决策和执行完全解耦。

引导文案：模板渲染

1. 引导 Prompt 模板

歧义判定通过后，接下来要生成引导文案。模板文件 guidance-prompt.st 很简洁：

```
关于{topic_name}，在知识库中检索到了以下内容：
{options}

请问你具体想了解哪个？请回复数字选择（可多选，如 1,2），或回复"都/全部"
```

两个占位符：{topic_name} 是触发歧义的主题名称，{options} 是品类列表。

2. buildPrompt 和 renderOptions

```
private String buildPrompt(String topicName, List<NodeScore> ranked) {
    String options = renderOptions(ranked);
}
```

```
return promptTemplateLoader.render(
    RAGConstant.GUIDANCE_PROMPT_PATH,
    Map.of(
        "topic_name", StrUtil.blankToDefault(topicName, ""),
        "options", options
    )
);
}
```

renderOptions 把候选列表转成带编号的文本，每个选项通过 resolveOptionDisplay 展示完整路径：

```
private String renderOptions(List<NodeScore> ranked) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < ranked.size(); i++) {
        IntentNode node = ranked.get(i).getNode();
        String display = resolveOptionDisplay(node);
        sb.append(i + 1).append(" ").append(display).append("\n");
    }
    return sb.toString().trim();
}

private String resolveOptionDisplay(IntentNode node) {
    if (node == null) {
        return "";
    }
    if (StrUtil.isNotBlank(node.getFullPath())) {
        return node.getFullPath();
    }
    return StrUtil.blankToDefault(node.getName(), node.getId());
}
```

resolveOptionDisplay 优先使用节点的 fullPath（如：OA系统 > 数据安全），让用户清楚看到系统归属和具体主题的层级关系。没有 fullPath 时降级为节点名称。
用多系统场景走一遍渲染，最终输出：

```
关于数据安全，在知识库中检索到了以下内容：
1) OA系统 > 数据安全管理
2) 保险系统 > 数据安全

请问你具体想了解哪个？请回复数字选择（可多选，如 1,2），或回复"都/全部"
```

用户一眼就能看清每个选项属于哪个系统、具体是什么主题，选择无歧义。

3. trimRankedOptions：选项数量兜底

```
private List<NodeScore> trimRankedOptions(List<NodeScore> ranked) {
    int maxOptions = Optional.ofNullable(guidanceProperties.getMaxOptions()).orElse(ranked.size())
    if (ranked.size() <= maxOptions) {
        return ranked;
    }
    return ranked.subList(0, maxOptions);
}
```



默认最多展示 6 个选项（maxOptions = 6）。电商平台品类再多——3C、家电、服装、食品、美妆、母婴、家居、运动——用户也不想看太长的列表。超出上限的按分数顺序截断（ranked 列表本身已按分数降序排列）。

Pipeline 短路：复用 SSE 通道

1. handleGuidance 的 boolean 返回值

引导文案生成好了，怎么送到用户手里？回到 StreamChatPipeline 的 execute() 方法：

```
public void execute(StreamChatContext ctx) {
    loadMemory(ctx);
    rewriteQuery(ctx);
    resolveIntents(ctx);

    if (handleGuidance(ctx)) {
        return;    // 短路：已经把引导文案推给用户，不再做检索
    }
    if (handleSystemOnly(ctx)) {
        return;
    }

    RetrievalContext retrievalCtx = retrieve(ctx);
    if (handleEmptyRetrieval(ctx, retrievalCtx)) {
        return;
    }

    streamRagResponse(ctx, retrievalCtx);
}
```

handleGuidance 在意图解析（resolveIntents）之后、检索（retrieve）之前调用。看它的实现：

```
private boolean handleGuidance(StreamChatContext ctx) {
    GuidanceDecision decision = guidanceService.detectAmbiguity(
        ctx.getRewriteResult().rewrittenQuestion(),
        ctx.getSubIntents()
    );
    if (!decision.isPrompt()) {
        return false;
    }
    StreamCallback callback = ctx.getCallback();
    callback.onContent(decision.getPrompt());
    callback.onComplete();
    return true;
}
```

三行核心逻辑：

1. callback.onContent(decision.getPrompt())：把引导文案通过 SSE 回调推给前端
2. callback.onComplete()：关闭 SSE 连接
3. return true：告诉 Pipeline 已处理完毕

Pipeline 拿到 true 就知道本轮请求已经结束了，立即 return，后面的 handleSystemOnly、retrieve、streamRagResponse 全部跳过。

2. SSE 事件流完全复用

引导文案的 SSE 事件流长这样：

event: meta	→ {conversationId, taskId}
event: message	→ {"type": "response", "delta": "关于退货政策，在知识..."}
event: message	→ {"type": "response", "delta": "...库中检索到了以下内容：\n1) 3C 数码\n..."}
event: finish	→ {completion payload}
event: done	→ [DONE]

和正常检索回复的事件流**完全一样**。没有专门的 guidance 事件类型，前端代码不需要做任何特殊适配——它只管接收 message 事件、拼接文本、展示给用户，不用关心这段文本是 LLM 生成的还是引导文案。

这个设计很巧妙。引导文案走和普通回复一样的 onContent + onComplete，前端视角看到的就是一段普通的回复文本。唯一的区别是回复速度——引导文案是预先拼好的字符串，onContent 一次推完，不需要等 LLM 逐 token 生成，响应几乎是瞬时的。

3. 短路带来的成本节省

对比引导短路和正常流程的开销：

阶段	正常流程	引导短路
会话记忆加载	✔ 已执行	✔ 已执行
查询改写	✔ 已执行	✔ 已执行
意图分类	✔ 已执行	✔ 已执行
歧义检测	跳过	✔ 已执行（多数纯内存，边界场景调一次 LLM）
向量检索 + BM25	✔ 已执行	✘ 跳过
Reranker 重排序	✔ 已执行	✘ 跳过
LLM 生成（~1~2s）	✔ 已执行	✘ 跳过

歧义检测在多数情况下是纯内存操作——分数比值明确时直接判定，代价可以忽略不计。只有分数落在灰色地带时才会调一次轻量 LLM（低 Temperature、短 Prompt），延迟约 200~500ms。但即使算上这个开销，它短路掉的部分——向量检索的数据库 IO、Reranker 的 Cross-Encoder 推理、LLM 的 token 生成——才是真正耗时和耗钱的大头。

4. 用户选择后的闭环

引导文案发出后，本轮 SSE 连接关闭，这一轮对话结束。接下来是用户的操作。用户看到引导消息，回复 1 或 3C 数码 或 全部。这个回复是一条新的用户消息，触发新的 SSE 会话——又走一遍完整的 Pipeline。不同的是，这次有了上一轮的上下文：

- 1. 会话记忆（第 2 篇） 把上一轮的引导文案和用户选择作为历史消息加载进来
- 2. 查询改写（第 4 篇） 基于历史上下文把 1 改写成 3C 数码的退货政策
- 3. 改写后的问题再走意图分类。多数情况下 3C 的分数会被拉高，分数比值低于边界下限，shouldSkipGuidance 直接拦住
- 4. 即使分类器没拉开分差，shouldSkipGuidance 的字符串匹配也会发现改写后的问题里包含"3C 数码"，触发快速跳过
- 5. 两道防线确保不再引导，继续走正常的检索和生成流程

这一闭环体现了 RAG 系统里各个环节的协同：歧义引导只负责问，会话记忆和查询改写负责把用户的选择拼回上下文，智能跳过负责避免重复引导。每个环节各司其职，串起来就是一个完整的交互循环。

小结

回顾本篇的核心要点：

- 1. 随着电商品类扩展，意图树自然长出跨品类的分支，歧义引导是必然需求。
- 2. 歧义引导只在单子问题场景触发，跨子问题的分数对比没有业务意义。
- 3. 按品类分组取最强代表，再用分数比值做分层判定——明确歧义区间直接触发，灰色地带调 LLM 确认，明确无歧义直接跳过。
- 4. shouldSkipGuidance 两条快速通道：分数比值低于边界下限、用户问题中显式提到品类名，任一命中即跳过。
- 5. resolveSystemNodeId 从叶子节点沿 parentId 向上回溯，在 CATEGORY 级停下——这一级是品类粒度，DOMAIN 太粗，TOPIC 太细。
- 6. AmbiguityLLMChecker 处理灰色地带：低 Temperature 保证确定性，降级策略选择触发引导（宁可多问不可答错），Prompt 设计覆盖领域线索识别和同义词识别两类规则方案的盲区。
- 7. GuidanceDecision 二态结构（NONE / PROMPT）让决策和执行解耦，Pipeline 层只需要调 isPrompt() 就知道怎么处理。
- 8. handleGuidance 返回 true 时 Pipeline 短路，复用 SSE 通道发送引导文案，前端零适配，省掉了检索和 LLM 生成的全部开销。

歧义引导走完了——要么用户被问了一次做出了选择，要么意图本身明确不需要引导。不管哪种情况，到这一步意图分类的结果已经确定了。下一个问题是：这些意图要怎么转化成实际的检索动作？KB 意图走哪个 Collection？topK 怎么定？MCP 意图怎么触发工具调用？SYSTEM 意图能直接短路回复吗？如果所有意图分数都太低，要不要兜底去全局搜一下？下一篇讲——意图分数出来了，该查哪个库、查多少条？

